

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

Grobentwurf für das Warenwirtschaftssystem (WAWI)

Software-Praktikum
Bachelor Technische Informatik
TH Köln

Inhaltsverzeichnis

1	Speicherstruktur der Daten	2
2	Systemarchitektur	4
3	Komponentenspezifikationen	6
3.1	BootLoader	6
3.2	ComponentController	6
3.3	WAWIDBModel	7
3.4	AdminGUI	9
3.5	AdminSteuerung	9
3.6	ProduktVerwaltung	11
3.7	LagerhalterGUI	13
3.8	LagerhalterSteuerung	14
3.9	LagerVerwaltung	16
3.10	SachbearbeiterGUI	18
3.11	SachbearbeiterSteuerung	19
3.12	BestellungVerwaltungS	21
3.13	KundeGUI	23
3.14	KundeSteuerung	24

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.15 KundeVerwaltung 26

3.16 BestellungVerwaltungK 27

Revisionshistorie

Vers.	Datum	Autor	Kommentare
6.0	20.03.2019	Budde	Finale Version für SWP im SoSe 2019

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

1 Speicherstruktur der Daten

Die konkreten Speicherstrukturen der Daten eines Systems werden typischerweise erst im Feinentwurf festgelegt.

Im Software-Praktikum legen wir diese jedoch bereits im Grobentwurf fest, weil diese Strukturen für alle Teilnehmer vorgegeben sind und somit alle Teilnehmer auf den gleichen Datenstrukturen Ihren eigenen Feinentwurf durchführen können. (Achtung: Nicht im Systementwurfs-Praktikum nachmachen!)

Die Abbildung 1.1 zeigt die konkrete Umsetzung der Datenspezifikation aus der Systemspezifikation [2] in der WAWI-Datenbank.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

2 Systemarchitektur

In Abbildung 2.1 sind alle Komponenten des WAWI-Systems zusammen mit ihren angebotenen und benutzten Schnittstelle zu erkennen. Eine Komponente im Softwarepraktikum entspricht einer Ansammlung von Java-Klassen, die in einer *.jar-Datei gepackt sind.

Für die Implementierung einer Komponente gilt allgemein: Wenn eine Komponente A eine Schnittstelle einer anderen Komponente B verwendet, ist Komponente A von Komponente B abhängig. Deshalb muss Komponente A die Komponente B in ihren „CLASSPATH“ aufnehmen. Abbildung 2.1 zeigt an, welche Komponente im WAWI welche Schnittstellen verwendet und stellt somit die Abhängigkeiten der einzelnen Komponenten untereinander dar.

Im Falle des WAWI benutzt zum Beispiel die „AdminSteuerung“-Komponente die „ICRUDProdukt“-Schnittstelle der „ProduktVerwaltung“-Komponente (siehe Abbildung 2.1) und implementiert die „IActivateComponent“-Schnittstelle der „ComponentController“-Komponente. Demzufolge müssen beide Komponenten („ProduktVerwaltung“ und „ComponentController“) in den „CLASSPATH“ der „AdminSteuerung“-Komponente aufgenommen werden.

Projekt:	Warenwirtschaftssystem	
Dokumenttyp:	Grobentwurf	Dokumentnr.: 410-WAWI 1
Datum:	20.03.2019	Version: 6.0

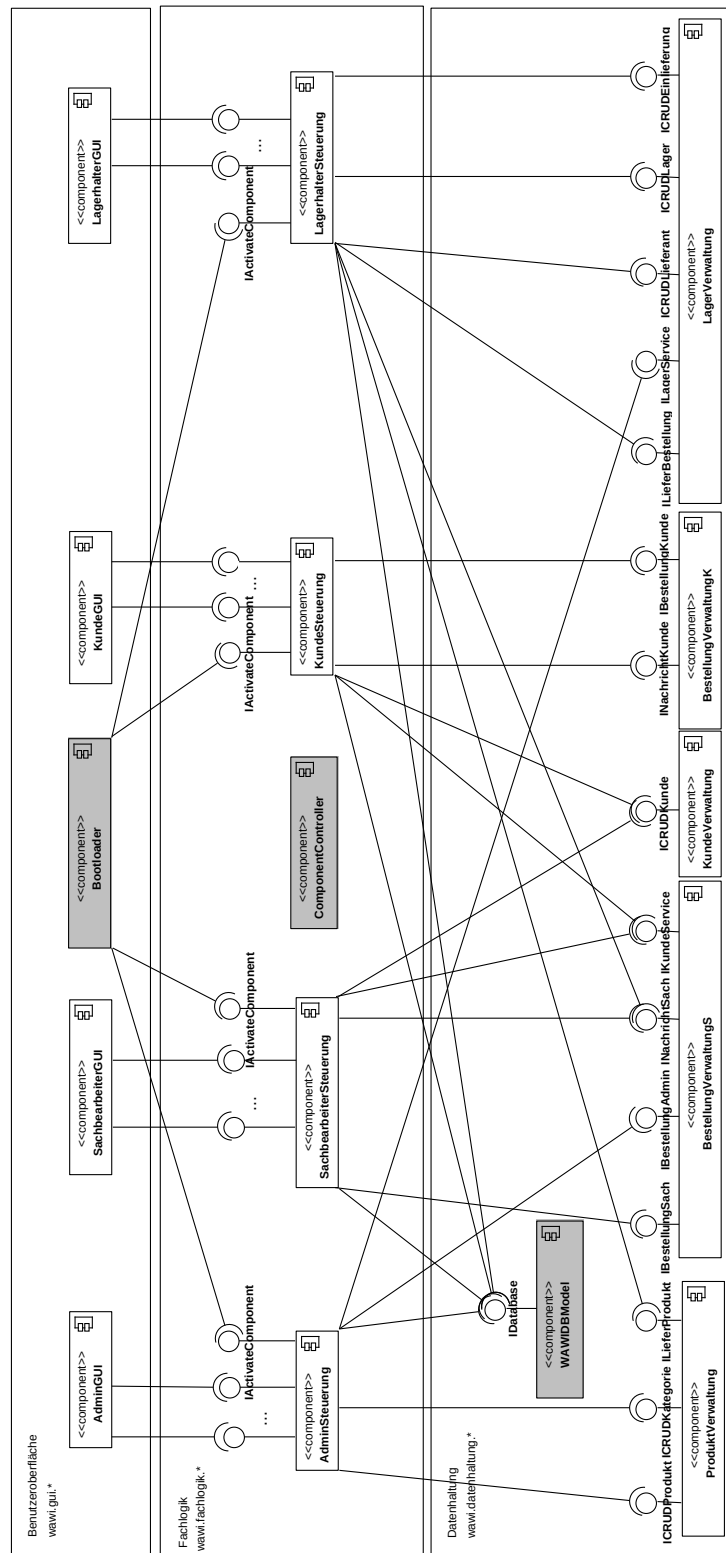


Abbildung 2.1: WAWI Komponenten-Diagramm mit allen Schnittstellen

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3 Komponentenspezifikationen

Dieses Kapitel spezifiziert die Funktionen, die Schnittstellen und das Verhalten der Komponenten des WAWI-Systems.

3.1 BootLoader

3.1.1 Funktionen

Die BootLoader-Komponente stellt das zentrale Startprogramm des WAWI-Systems bereit. Es handelt sich dabei um eine Oberfläche, von der aus die speziellen Bedienoberflächen der Komponenten AdminGUI, SachbearbeiterGUI, LagerhalterGUI und KundeGUI gestartet werden können. Der Start der speziellen Bedienoberflächen dieser Komponenten geschieht durch die Verwendung der jeweiligen Implementierung der `IActivateComponent`-Schnittstelle.

Diese Komponente wird von den Organisatoren des Software-Praktikums implementiert.

3.1.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

3.1.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.2 ComponentController

3.2.1 Funktionen

Die ComponentController-Komponente stellt die Interface-Klasse `IActivateComponent` im Paket `wawi.fachlogik.componentcontroller.services` zur Verfügung. Diese muss von den Komponenten der Fachlogik-Schicht implementiert werden.

Diese Komponente wird von den Organisatoren des Software-Praktikums implementiert.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.2.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

3.2.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.3 WAWIDBModel

3.3.1 Funktionen

Die „WAWIDBModel“-Komponente besteht aus Entitätsklassen, welche die Tabellen oder Entitäten der Datenbank des Warenwirtschaftssystems repräsentieren wie sie in der Systemspezifikation spezifiziert sind. Es existieren zwei identische Instanzen der Datenbank des Warenwirtschaftssystems: „wawidb_dev“ (Development) und „wawidb_prod“ (Produktiv). Entsprechend dazu enthält die „WAWIDBModel“-Komponente auch zwei „Persistence-Units“ namens „WAWIDBDEVPU“ und „WAWIDBPRODPU“, mithilfe derer auf die entsprechende Instanz der Datenbank zugegriffen werden kann. Während der gesamten Entwicklungsphase muss die „wawidb_dev“-Datenbank verwendet werden, im Quellcode sollte die Persistence-Unit „WAWIDBDEVPU“ genutzt werden. Erst nach der erfolgreichen Integration des Gesamtsystems soll auf die „wawidb_prod“-Datenbank zugegriffen werden. Die „WAWIDBModel“-Komponente wurde bereits implementiert und kann aus dem Git-Repository ausgecheckt werden. Um auf die Datenbank zugreifen zu können, müssen alle Komponenten der Datenhaltung-Schicht die WAWIDBModel-Komponente und die MySQL JDBC Driver- und EclipseLink (JPA 2.0)-Bibliotheken unmittelbar verwenden.

Die Klassen der Entitäten wird von dieser Komponente im Paket `wawi.datenhaltung.wawidbmodel.entities` bereit gestellt.

Diese Komponente wird von den Organisatoren des Software-Praktikums implementiert.

3.3.2 Schnittstellen

IDatabase Die WAWIDBModel - Komponente bietet die IDatabase - Schnittstelle an. Diese wird von allen anderen Komponenten verwendet, um über einen systemweit einzigen EntityManager

Projekt:	Warenwirtschaftssystem		
Dokumenttyp:	Grobentwurf	Dokumentnr.:	410-WAWI 1
Datum:	20.03.2019	Version:	6.0

einen konsistenten Zugriff auf die Datenbank zu ermöglichen.

Die Schnittstelle ist in Abbildung ?? spezifiziert.

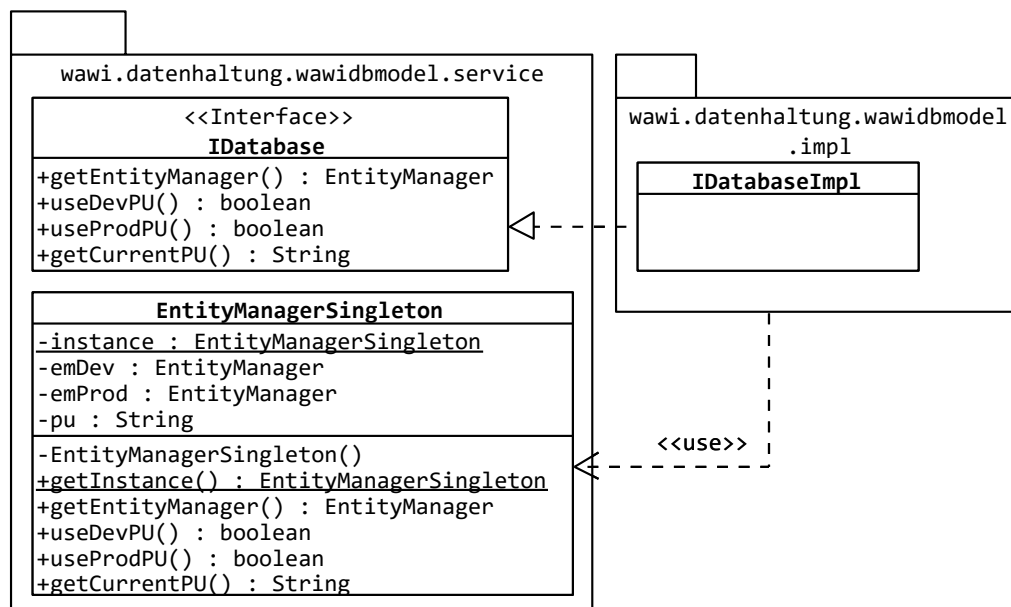


Abbildung 3.1: IDatabase - Schnittstelle der WAWIDBModel-Komponente

useDevPU wählt die Developer DB zur Benutzung durch die Persistence Unit aus.

useProdPU wählt die Produktiv DB zur Benutzung durch die Persistence Unit aus.

getCurrentPU liefert eine Information über die aktuell ausgewählte Persistence Unit (PU).

Der Rückgabewert ist vom Typ `String` und hat entweder den Wert `WAWIDEVPU` oder `WAWIPRODPU`.

Diese Schnittstelle ist bereits implementiert und muss von den Teilnehmern des SWP verwendet werden.

3.3.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.4 AdminGUI

Die Einstiegs-GUI-Klasse muß den Namen `FrameAdmin` haben und im Paket `wawi.gui.admingui.gui` liegen.

3.4.1 Funktionen

Die `AdminGUI` - Komponente stellt die Benutzeroberfläche für alle Anwendungsfälle des Admins zur Verfügung.

Der Feinentwurf dieser Komponente ist nicht vorgegeben und soll von dem `Admin` - Team eigenständig erstellt werden.

3.4.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

3.4.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.5 AdminSteuerung

3.5.1 Funktionen

Diese Komponente realisiert die Anwendungsfälle: `/LF100/`, `/LF105/`, `/LF110/`, `/LF120/`, `/LF130/` und `/LF140/`.

Die zu verwendenden Schnittstellen der Komponenten sind aus Diagramm 2.1 zu entnehmen. Es dürfen **keine** weiteren Schnittstellen benutzt werden.

3.5.2 Schnittstellen

IActivateComponent

Die `AdminSteuerung` - Komponente bietet eine Implementierung der `IActivateComponent` - Schnittstelle an, die von der `Bootloader` - Komponente verwendet wird, um auf ihre GUI zuzugreifen.

Projekt:	Warenwirtschaftssystem		
Dokumenttyp:	Grobentwurf	Dokumentnr.:	410-WAWI 1
Datum:	20.03.2019	Version:	6.0

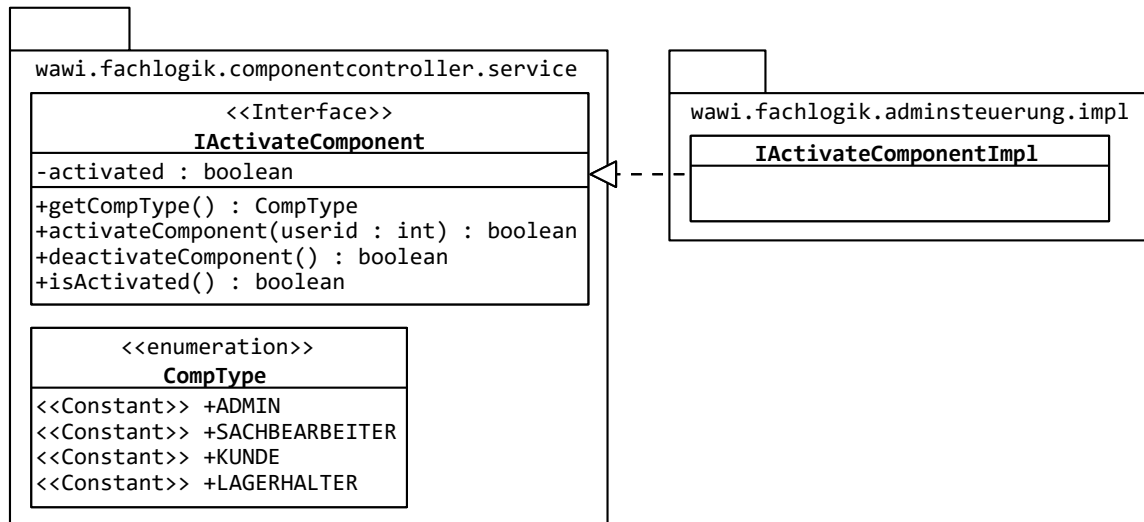


Abbildung 3.2: Implementierung der Schnittstelle IActivateComponent

Der Parameter der `activateComponent` - Methode der `IActivateComponent` - Schnittstelle enthält die jeweilige ID des Admins, für den die Komponente aktiviert werden soll. Falls eine falsche ID übergeben wird, sollte die Komponente nicht aktiviert werden. Die Komponente darf nur für den Default-Admin (`userid = 9`) aktiviert werden.

Zusätzliche Schnittstellen

Zusätzliche Schnittstellen der Komponente werden vom Admin-Team entworfen und erstellt. Die `AdminGUI`-Komponente soll diese und nur diese Schnittstellen verwenden.

3.5.3 Verhalten

Die Komponente soll die beiden Zustände *Aktiviert* und *Deaktiviert* annehmen können.

- Wird die Komponente gestartet, so befindet sie sich zunächst im Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* `activateComponent()` mit zulässiger ID aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Aktiviert*.
- Wird `activateComponent()` mit einer nicht zulässigen ID aufgerufen, wird nur `false` zurück gegeben.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

- Wird im Zustand *Aktiviert* wiederholt `activateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Aktiviert*.
- Wird im Zustand *Aktiviert* `deactivateComponent()` aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* wiederholt `deactivateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Deaktiviert*.

Siehe WAWI Testspezifikation für genauere Definition des erwarteten Verhaltens der Methoden dieser Schnittstelle.

3.6 ProduktVerwaltung

3.6.1 Funktionen

Diese Komponente realisiert selbst keine Anwendungsfälle und besitzt auch keine GUI. Sie ist eine Unterstützungs-Komponente und stellt den anderen Komponenten Schnittstellen-Operationen für die Umsetzung von Anwendungsfällen zur Verfügung.

Projekt:	Warenwirtschaftssystem		
Dokumenttyp:	Grobentwurf	Dokumentnr.:	410-WAWI 1
Datum:	20.03.2019	Version:	6.0

3.6.2 Schnittstellen

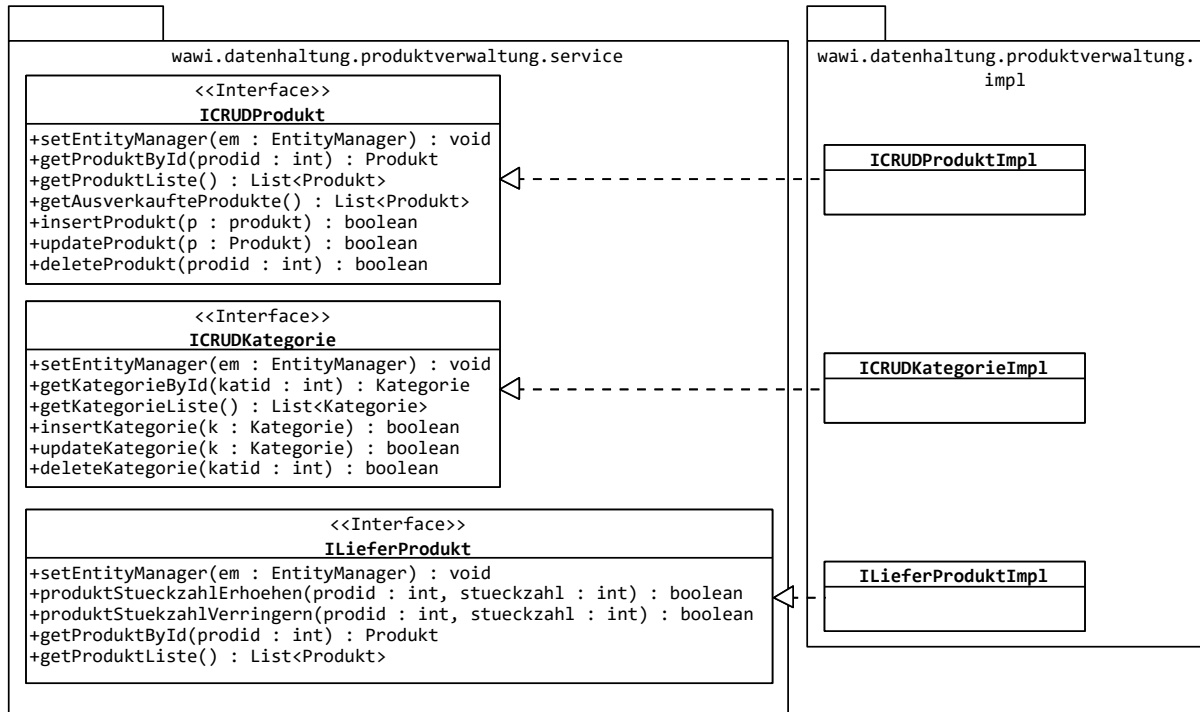


Abbildung 3.3: Interfaces der „ProduktVerwaltung“-Komponente

Die **CRUD Methoden** (insertXXX, updateXXX, deleteXXX, getXXX,) haben ihre dem Namen nach herkömmlichen Standardfunktionen.

ICRUDProdukt

getProduktListe() Der Lagerhalter soll in /LF20/ und /LF25/ Einlieferungen anlegen und bearbeiten. Die `getProduktListe()` Methode liefert ihm dazu eine Liste aller Produkte, unabhängig davon, ob diese aktiv (`aktiv=1`) oder inaktiv (`aktiv=0`) sind, um die gewünschten Produkte zu einer Einlieferung hinzuzufügen.

ILieferProdukt

produktStueckzahlErhoehen die Stückzahl eines über die ID bestimmten Produkts wird um die übergebene Stückzahl erhöht. War die Operation erfolgreich gibt die Methode ein `true` zurück, sonst `false`.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

produktStueckzahlVerringern die Stückzahl eines über die ID bestimmten Produkts wird um die übergebene Stückzahl verringert. War die Operation erfolgreich gibt die Methode ein `true` zurück, sonst `false`.

Eine detailliertere Beschreibung ist der Testspezifikation zu entnehmen.

3.6.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.7 LagerhalterGUI

Die Einstiegs-GUI-Klasse muß den Namen `FrameLagerhalter` haben und im Paket `wawi.gui.lagerhaltergui.gui` liegen.

3.7.1 Funktionen

Die `LagerhalterGUI` - Komponente stellt die Benutzeroberfläche für alle Anwendungsfälle des Lagerhalters zur Verfügung.

Der Feinentwurf dieser Komponente ist nicht vorgegeben und soll von dem `Lagerhalter` - Team eigenständig erstellt werden.

3.7.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

3.7.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.8 LagerhalterSteuerung

3.8.1 Funktionen

Diese Komponente realisiert die Anwendungsfälle: /LF10/, /LF15/, /LF20/, /LF25/, /LF27/, /LF30/, /LF40/, /LF45/.

Die zu verwendenden Schnittstellen der Komponenten sind aus Diagramm 2.1 zu entnehmen. Es dürfen **keine** weiteren Schnittstellen benutzt werden.

3.8.2 Schnittstellen

IActivateComponent

Die LagerhalterSteuerung - Komponente bietet eine Implementierung der IActivateComponent - Schnittstelle an, die von der Bootloader - Komponente verwendet wird, um auf ihre GUI zuzugreifen.

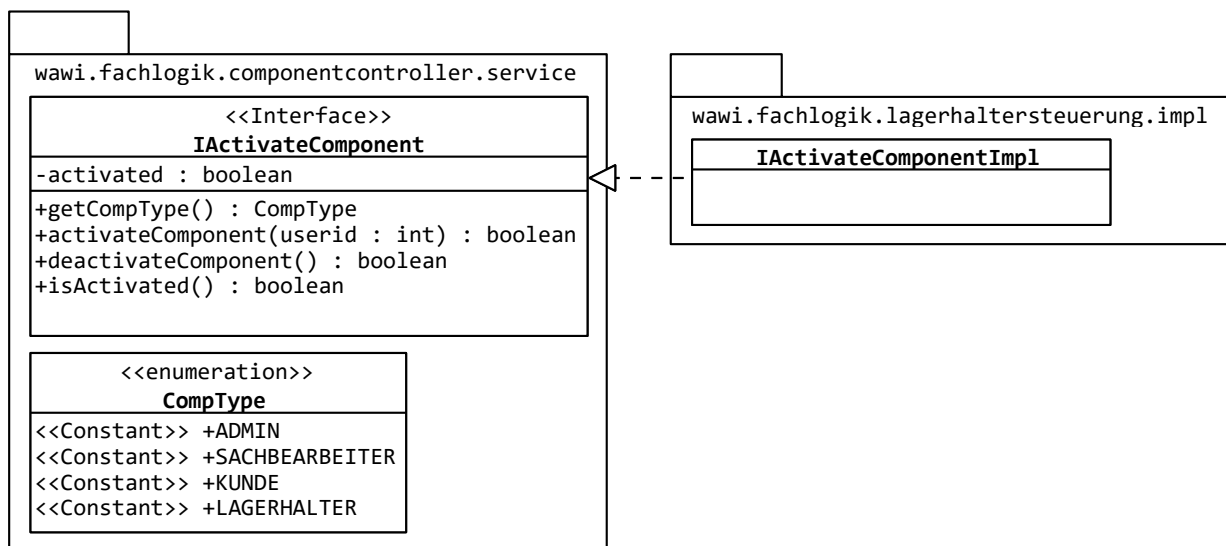


Abbildung 3.4: Implementierung der Schnittstelle IActivateComponent

Der Parameter der activateComponent - Methode der IActivateComponent - Schnittstelle enthält die jeweilige ID des Lagerhalters, für den die Komponente aktiviert werden soll. Falls eine falsche ID übergeben wird, sollte die Komponente nicht aktiviert werden. Die Komponente darf nur für den Default-Lagerhalter (userid = 10) aktiviert werden.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

Zusätzliche Schnittstellen

Zusätzliche Schnittstellen der Komponente werden vom Lager-Team entworfen und erstellt. Die LagerGUI-Komponente soll diese und nur diese Schnittstellen verwenden.

3.8.3 Verhalten

Die Komponente soll die beiden Zustände *Aktiviert* und *Deaktiviert* annehmen können.

- Wird die Komponente gestartet, so befindet sie sich zunächst im Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* `activateComponent()` mit zulässiger ID aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Aktiviert*.
- Wird `activateComponent()` mit einer nicht zulässigen ID aufgerufen, wird nur `false` zurück gegeben.
- Wird im Zustand *Aktiviert* wiederholt `activateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Aktiviert*.
- Wird im Zustand *Aktiviert* `deactivateComponent()` aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* wiederholt `deactivateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Deaktiviert*.

Siehe WAWI Testspezifikation für genauere Definition des erwarteten Verhaltens der Methoden dieser Schnittstelle.

Für /LF20/ sind mehrere Fälle zu unterscheiden und zu beachten:

Löschen einer Lieferposition

- Wenn zu löschende Lieferpositon einzige Lieferposition aus einem Lager ist, dann Lagerverkehr auf del setzen
- Stückzahl in Produkt reduzieren
- Lieferposition löschen

Lieferposition hinzufügen

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

- Hinzufügen einer Lieferposition
- Stückzahl in Produkt erhöhen
- Falls noch kein Lagerverkehr mit Produkt aus diesem Lager existiert, neuen Lagerverkehr anlegen

Einlieferung komplett löschen

- Alle Produkte um gelieferte Anzahl reduzieren
- Löschen der Lieferpositionen
- Setzen der zugehörigen Lagerverkehre auf Status del
- Löschen der Einlieferung

Die Fälle könne auch kombiniert miteinander auftreten.

3.9 LagerVerwaltung

3.9.1 Funktionen

Diese Komponente realisiert selbst keine Anwendungsfälle und besitzt auch keine GUI. Sie ist eine Unterstützungs-Komponente und stellt den anderen Komponenten Schnittstellen-Operationen für die Umsetzung von Anwendungsfällen zur Verfügung.

Projekt:	Warenwirtschaftssystem	
Dokumenttyp:	Grobentwurf	Dokumentnr.: 410-WAWI 1
Datum:	20.03.2019	Version: 6.0

3.9.2 Schnittstellen

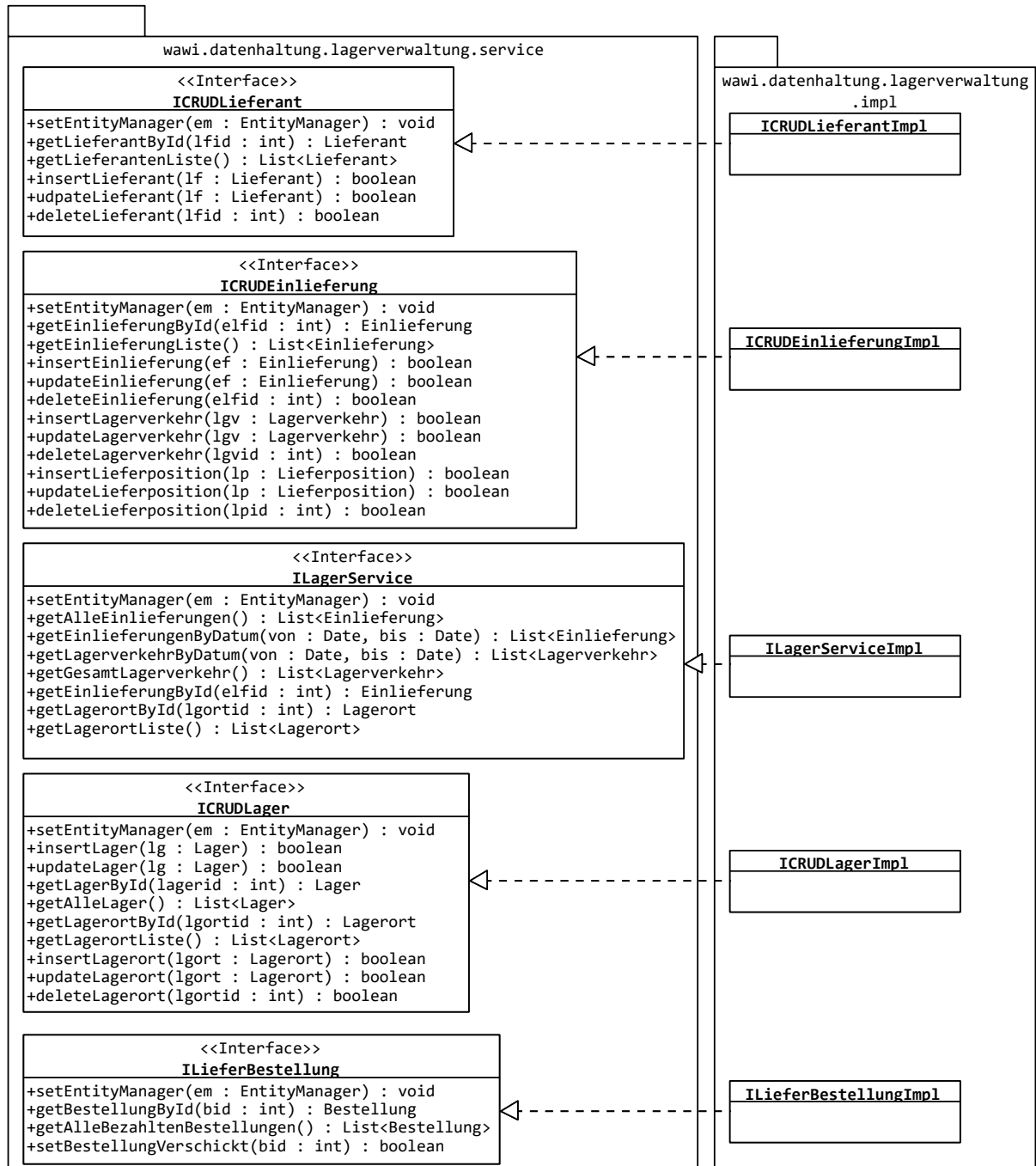


Abbildung 3.5: Interfaces der „LagerVerwaltung“-Komponente

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

Die meisten Operationen erklären ihre Bedeutung bereits durch ihre Namen und Parameter. Eine detailliertere Beschreibung ist der Testspezifikation zu entnehmen.

Die **CRUD Methoden** (insertXXX, updateXXX, deleteXXX, getXXX,) haben ihre dem Namen nach herkömmlichen Standardfunktionen.

ILagerService

getEinlieferungByDatum liefert alle Einlieferungen die im übergebenen Datumsintervall liegen inklusive des Start- und Enddatums.

ILieferBestellung

setBestellungVerschickt markiert eine Bestellung als verschickt in der Datenbank.

3.9.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.10 SachbearbeiterGUI

Die Einstiegs-GUI-Klasse muß den Namen `FrameSachbearbeiter` haben und im Paket `wawi.gui.sachbearbeitergui.gui` liegen.

3.10.1 Funktionen

Die `SachbearbeiterGUI` - Komponente stellt die Benutzeroberfläche für alle Anwendungsfälle des Lagerhalters zur Verfügung.

Der Feinentwurf dieser Komponente ist nicht vorgegeben und soll von dem Lagerhalter - Team eigenständig erstellt werden.

3.10.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.10.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.11 SachbearbeiterSteuerung

3.11.1 Funktionen

Diese Komponente realisiert die Anwendungsfälle: /LF50/, /LF60/, /LF65/, /LF70/, /LF80/, /LF90/.

Die zu verwendenden Schnittstellen der Komponenten sind aus Diagramm 2.1 zu entnehmen. Es dürfen **keine** weiteren Schnittstellen benutzt werden.

3.11.2 Schnittstellen

IActivateComponent

Die SachbearbeiterSteuerung - Komponente bietet eine Implementierung der IActivateComponent - Schnittstelle an, die von der Bootloader - Komponente verwendet wird, um auf ihre GUI zuzugreifen.

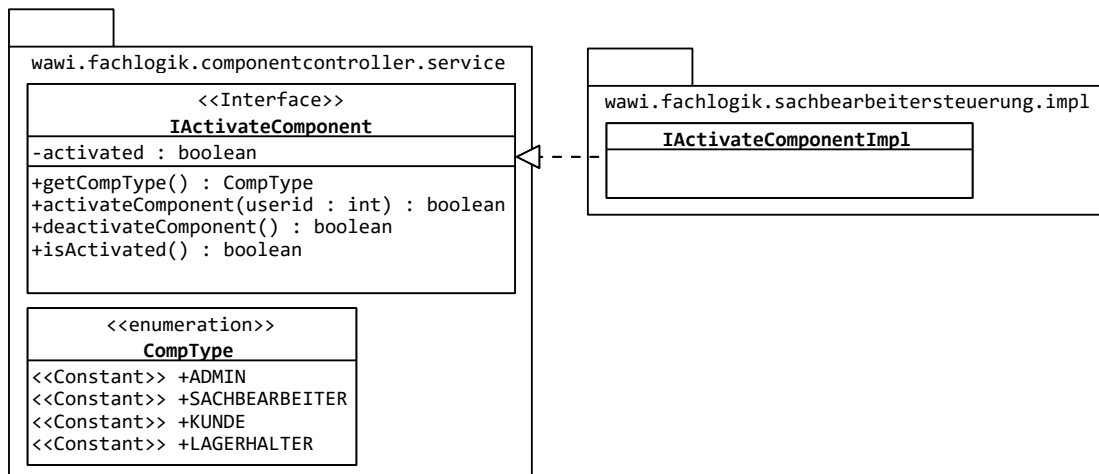


Abbildung 3.6: Implementierung der Schnittstelle IActivateComponent

Der Parameter der activateComponent - Methode der IActivateComponent - Schnittstelle enthält die jeweilige ID des Sachbearbeiters, für den die Komponente aktiviert werden soll.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

Falls eine falsche ID übergeben wird, sollte die Komponente nicht aktiviert werden. Die Komponente darf nur für den Default-Sachbearbeiter (*userid* = 20) aktiviert werden.

Zusätzliche Schnittstellen

Zusätzliche Schnittstellen der Komponente werden vom *Sachbearbeiter*-Team entworfen und erstellt. Die *SachbearbeiterGUI*-Komponente soll diese und nur diese Schnittstellen verwenden.

3.11.3 Verhalten

Die Komponente soll die beiden Zustände *Aktiviert* und *Deaktiviert* annehmen können.

- Wird die Komponente gestartet, so befindet sie sich zunächst im Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* `activateComponent()` mit zulässiger ID aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Aktiviert*.
- Wird `activateComponent()` mit einer nicht zulässigen ID aufgerufen, wird nur `false` zurück gegeben.
- Wird im Zustand *Aktiviert* wiederholt `activateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Aktiviert*.
- Wird im Zustand *Aktiviert* `deactivateComponent()` aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* wiederholt `deactivateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Deaktiviert*.

Siehe WAWI Testspezifikation für genauere Definition des erwarteten Verhaltens der Methoden dieser Schnittstelle.

Für /LF60/ und /LF65/ sind mehrere Fälle zu unterscheiden und zu beachten:

/LF60/ Bestellung etwas hinzufügen

- Hinzufügen neue(r) Bestellungsposition(en)
- Anpassen von Gesamtnetto und Brutto in Bestellung

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

/LF65/ Position entfernen / stornieren

Position entfernen

- Löschen von bestellungsposition(en)
- Anpassen von Gesamtnetto und Brutto in Bestellung

Bestellung stornieren

- Überprüfen des Status ob Bestellung storniert werden darf
- Bestellung auf storniert setzen

Die Fälle könne auch kombiniert miteinander auftreten.

3.12 BestellungVerwaltungS

3.12.1 Funktionen

Diese Komponente realisiert selbst keine Anwendungsfälle und besitzt auch keine GUI. Sie ist eine Unterstützungs-Komponente und stellt den anderen Komponenten Schnittstellen-Operationen für die Umsetzung von Anwendungsfällen zur Verfügung.

Projekt:	Warenwirtschaftssystem		
Dokumenttyp:	Grobentwurf	Dokumentnr.:	410-WAWI 1
Datum:	20.03.2019	Version:	6.0

3.12.2 Schnittstellen

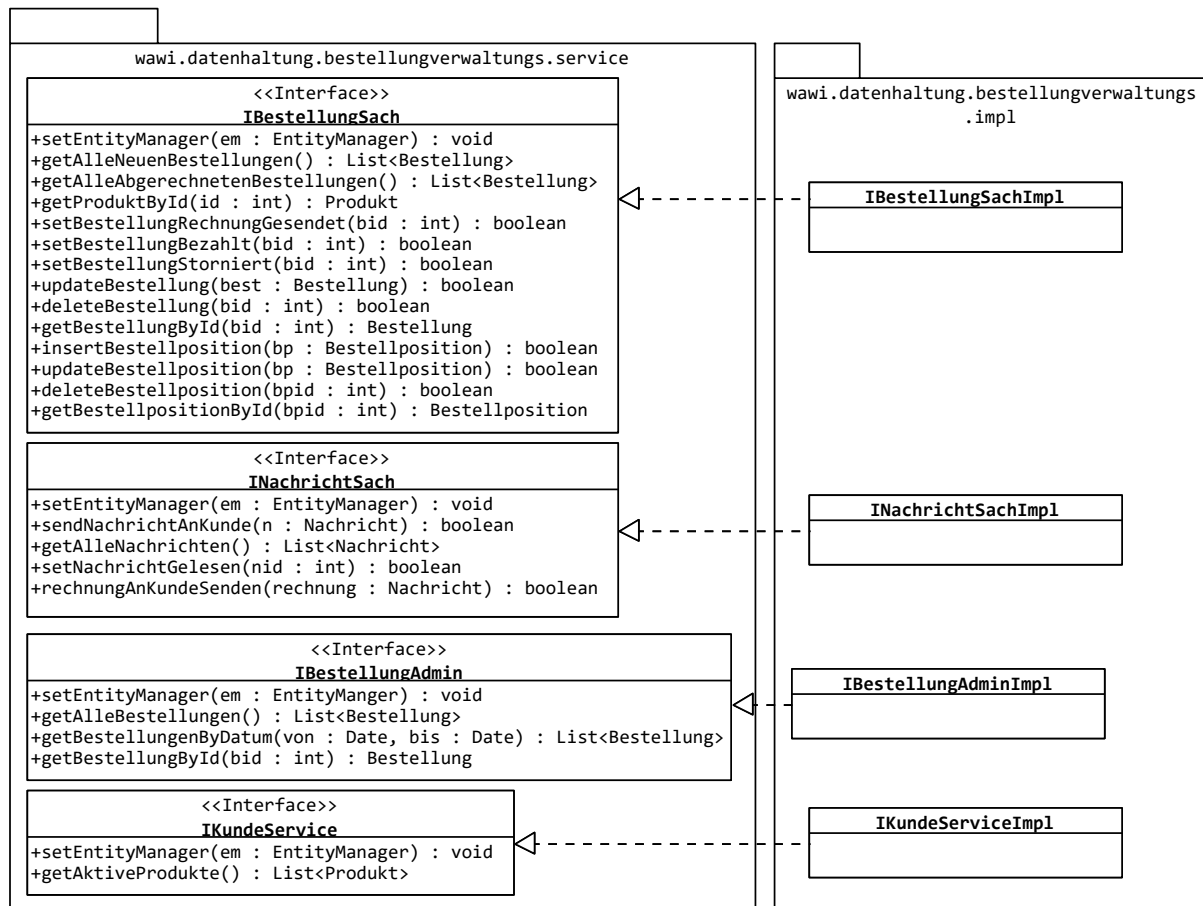


Abbildung 3.7: Interfaces der „BestellungsverwaltungS“-Komponente

Die meisten Operationen erklären ihre Bedeutung bereits durch ihre Namen und Parameter. Eine detailliertere Beschreibung ist der Testspezifikation zu entnehmen.

Die **CRUD Methoden** (insertXXX, updateXXX, deleteXXX, getXXX,) haben ihre dem Namen nach herkömmlichen Standardfunktionen.

IBestellungSach

setBestellung***()** setzt den entsprechenden Status (Gesendet/Bezahlt/Storniert) in der DB.

INachrichtSach

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

getAlleNachrichten() Der Sachbearbeiter soll in /LF90/ Nachrichten von Kunden anzeigen lassen, um diese zu beantworten. Die `getAlleNachrichten()` Methode liefert die dazu erforderliche Liste aller Nachrichten die von Kunden an das WAWI (`typ = 'anwawi'`) gesendet wurden.

sendNachrichtAnKunde() In /LF90/ soll es dem Sachbearbeiter möglich sein eine Nachricht eines Kunden zu beantworten. Die an die `sendNachrichtAnKunde()` Methode übergebene Nachricht muss dabei an den Kunden adressiert (gleiche `kid` wie zu beantwortende Nachricht und `typ = 'ankunde'`) und ungelesen sein (`status = 'ungelesen'`).

rechnungAnKundeSenden() Der Sachbearbeiter soll in /LF70/ Bestellungen abrechnen und eine Rechnung an den Kunden schicken. Die an die `rechnungAnKundeSenden()` Methode übergebene Nachricht muss an den Kunden adressiert (gleiche `kid` wie abzurechnende Bestellung und `typ = 'ankunde'`) und ungelesen sein (`status = 'ungelesen'`). Sie enthält als `nachricht` die Rechnung. Die Rechnung soll zusätzlich im Dateisystem gespeichert werden.

3.12.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.13 KundeGUI

Die Einstiegs-GUI-Klasse muß den Namen `FrameKunde` haben und im Paket `wawi.gui.kundegui.gui` liegen.

3.13.1 Funktionen

Die `KundGUI` - Komponente stellt die Benutzeroberfläche für alle Anwendungsfälle des Kunden zur Verfügung.

Der Feinentwurf dieser Komponente ist nicht vorgegeben und soll von dem Kunden - Team eigenständig erstellt werden.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.13.2 Schnittstellen

Diese Komponente bietet keine Schnittstellen für andere Komponenten an.

3.13.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.14 KundeSteuerung

3.14.1 Funktionen

Diese Komponente realisiert die Anwendungsfälle: /LF150/, /LF160/ und /LF170/.

Die zu verwendenden Schnittstellen der Komponenten sind aus Diagramm 2.1 zu entnehmen.
Es dürfen **keine** weiteren Schnittstellen benutzt werden.

3.14.2 Schnittstellen

IActivateComponent

Die KundeSteuerung - Komponente bietet eine Implementierung der IActivateComponent - Schnittstelle an, die von der Bootloader - Komponente verwendet wird, um auf ihre GUI zuzugreifen.

Projekt:	Warenwirtschaftssystem		
Dokumenttyp:	Grobentwurf	Dokumentnr.:	410-WAWI 1
Datum:	20.03.2019	Version:	6.0

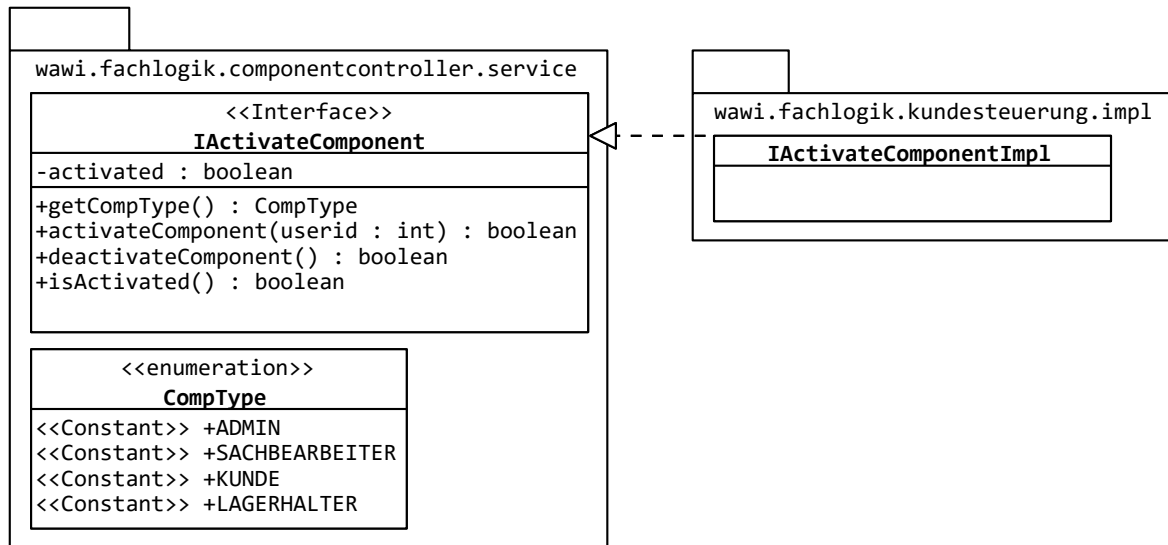


Abbildung 3.8: Implementierung der Schnittstelle IActivateComponent

Der Parameter der `activateComponent` - Methode der `IActivateComponent` - Schnittstelle enthält die ID eines Kunden, für den die Komponente aktiviert werden soll. Falls eine falsche ID übergeben wird, sollte die Komponente nicht aktiviert werden. Die Komponente darf nur für einen in der Datenbank existierenden Kunden aktiviert werden.

Zusätzliche Schnittstellen

Zusätzliche Schnittstellen der Komponente werden vom Kunde-Team entworfen und erstellt. Die `KundeGUI`-Komponente soll diese und nur diese Schnittstellen verwenden.

3.14.3 Verhalten

Die Komponente soll die beiden Zustände *Aktiviert* und *Deaktiviert* annehmen können.

- Wird die Komponente gestartet, so befindet sie sich zunächst im Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* `activateComponent()` mit zulässiger ID aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Aktiviert*.
- Wird `activateComponent()` mit einer nicht zulässigen ID aufgerufen, wird nur `false` zurück gegeben.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

- Wird im Zustand *Aktiviert* wiederholt `activateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Aktiviert*.
- Wird im Zustand *Aktiviert* `deactivateComponent()` aufgerufen, gibt die Methode ein `true` zurück und geht in den Zustand *Deaktiviert*.
- Wird im Zustand *Deaktiviert* wiederholt `deactivateComponent()` aufgerufen, wird ein `false` zurück gegeben, die Komponenten bleibt im Zustand *Deaktiviert*.

Siehe WAWI Testspezifikation für genauere Definition des erwarteten Verhaltens der Methoden dieser Schnittstelle.

3.15 KundeVerwaltung

3.15.1 Funktionen

Diese Komponente realisiert selbst keine Anwendungsfälle und besitzt auch keine GUI. Sie ist eine Unterstützungs-Komponente und stellt den anderen Komponenten Schnittstellen-Operationen für die Umsetzung von Anwendungsfällen zur Verfügung.

3.15.2 Schnittstellen

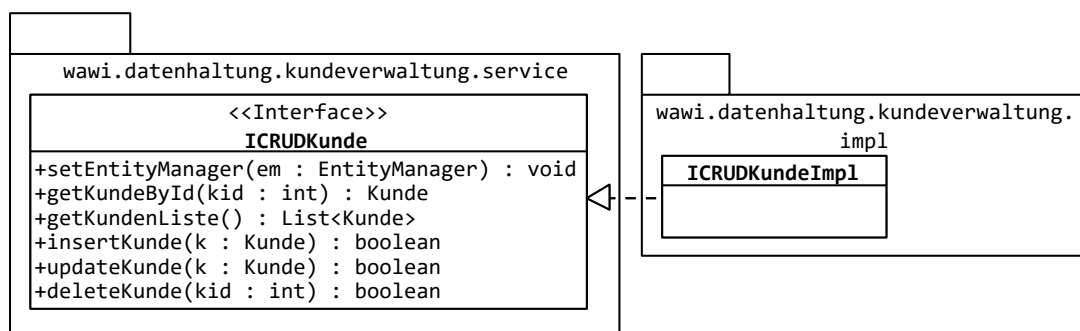


Abbildung 3.9: Interfaces der „KundeVerwaltung“-Komponente

Die meisten Operationen erklären ihre Bedeutung bereits durch ihre Namen und Parameter. Eine detailliertere Beschreibung ist der Testspezifikation zu entnehmen.

Die **CRUD Methoden** (`insertXXX`, `updateXXX`, `deleteXXX`, `getXXX`,) haben ihre dem Namen nach herkömmlichen Standardfunktionen.

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

3.15.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

3.16 BestellungVerwaltungK

3.16.1 Funktionen

Diese Komponente realisiert selbst keine Anwendungsfälle und besitzt auch keine GUI. Sie ist eine Unterstützungs-Komponente und stellt den anderen Komponenten Schnittstellen-Operationen für die Umsetzung von Anwendungsfällen zur Verfügung.

3.16.2 Schnittstellen

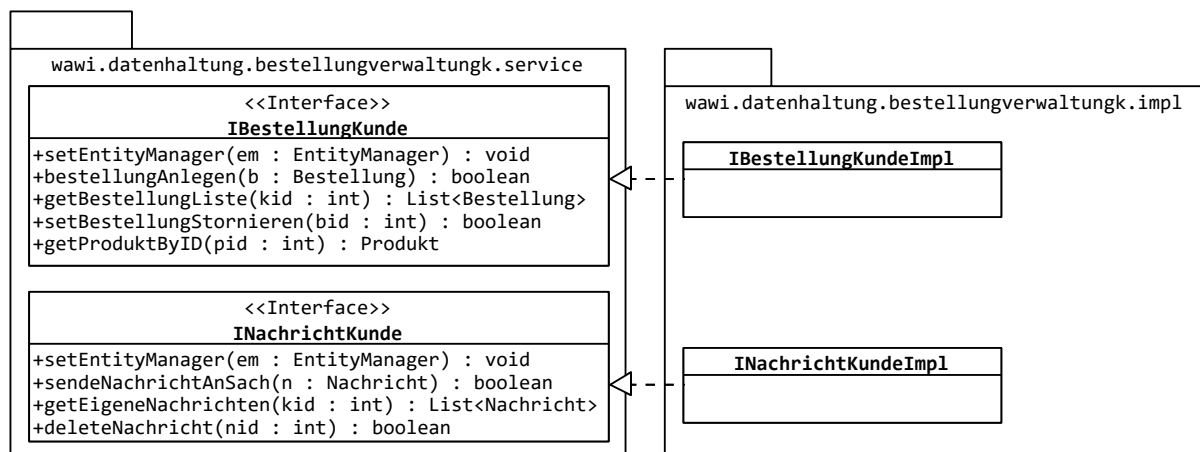


Abbildung 3.10: Interfaces der „BestellungVerwaltungK“-Komponente

Die meisten Operationen erklären ihre Bedeutung bereits durch ihre Namen und Parameter. Eine detailliertere Beschreibung ist der Testspezifikation zu entnehmen.

Die **CRUD Methoden** (insertXXX, updateXXX, deleteXXX, getXXX,) haben ihre dem Namen nach herkömmlichen Standardfunktionen.

InachrichtKunde

bestellungAnlegen() Der Kunde soll in /LF150/ eine Bestellung anlegen können. Die

<i>Projekt:</i>	Warenwirtschaftssystem		
<i>Dokumenttyp:</i>	Grobentwurf	<i>Dokumentnr.:</i>	410-WAWI 1
<i>Datum:</i>	20.03.2019	<i>Version:</i>	6.0

an die `bestellungAnlegen()` Methode übergebene Bestellung muss neu sein (`status = 'n'`).

sendeNachrichtAnSach() In /LF170/ soll es dem Kunden möglich sein eine Nachricht an den Sachbearbeiter zu senden. Die an die `sendeNachrichtAnSach()` Methode übergebene Nachricht muss dabei vom Kunden stammen (`kid` des Kunden), an den Sachbearbeiter adressiert sein (`typ = 'anwawi'`) und ungelesen sein (`status = 'ungelesen'`).

3.16.3 Verhalten

Diese Komponente besitzt kein zu spezifizierendes Verhalten.

Literatur

- [1] Software-Praktikum, *Lastenheft für das Warenwirtschaftssystem (WAWI)*. TH Köln
- [2] Software-Praktikum, *Systemspezifikation für das Warenwirtschaftssystem (WAWI)*. TH Köln
- [3] Software-Praktikum, *Testspezifikation für das Warenwirtschaftssystem (WAWI)*. TH Köln